

NovaSMS

Mode Configuration — Providers, Opérateurs & Résilience

Pattern Strategy/Factory déjà en place · Comment basculer d'un fournisseur ou d'un opérateur à l'autre · Circuit Breaker

Stage Sankofa Lab · Romuald KO · Document régénéré depuis le code réel du dépôt
Plateforme SaaS B2B Messagerie Multi-Canal — NestJS · PostgreSQL · Redis · React 19

Table des matières

1	Pourquoi un mode configuration
2	Le pattern appliqué — Strategy + Factory
3	Canal SMS — SMS_PROVIDER
4	Canal Email — EMAIL_PROVIDER
5	Canal WhatsApp — WHATSAPP_PROVIDER
6	Paiement — MOBILE_MONEY_PROVIDER & VISA_PROVIDER
7	Opérateurs Mobile Money — règles par opérateur
8	Stockage fichiers — détection automatique S3 / local
9	Notifications Push — PUSH_PROVIDER (non branché)
10	Résilience — le Circuit Breaker maison
11	Procédure pratique — basculer simulation !' production
12	Récapitulatif — toutes les variables de configuration

1 — Pourquoi un mode configuration

NovaSMS s'intègre à de nombreux services externes (SMS, Email, WhatsApp, paiement carte, paiement Mobile Money, stockage fichiers, notifications push), chacun ayant plusieurs fournisseurs concurrents possibles selon le marché ou le budget du client final. Plutôt que de coder en dur un fournisseur par canal, chaque famille de service est accédée exclusivement à travers une interface commune (ex. SmsProvider, EmailProvider), et une Factory se charge de choisir puis d'instancier l'implémentation concrète à utiliser, en lisant uniquement des variables d'environnement.

%Æ Principe fondamental (commenté explicitement dans le code)

« STAGING ET PRODUCTION = EXACTEMENT LE MEME CODE, seul le .env diffère. »

Changer de fournisseur ou d'opérateur ne nécessite donc jamais de modification de code ni de nouveau déploiement applicatif — uniquement une variable d'environnement et un redémarrage du service.

2 — Le pattern appliqué — Strategy + Factory

4

Chaque famille de fournisseurs suit la même structure en trois couches, illustrée ici sur l'exemple SMS mais identique pour Email, WhatsApp et Paiement.

providers/sms/ — structure Strategy (interface) + Factory (sélection)

```
1 interface SmsProvider {
2   send(to: string, message: string): Promise<{ success: boolean }>;
3   sendBatch(contacts, message): Promise<{ sent: number; failed: number }>;
4 }
5
6 class TwilioProvider implements SmsProvider { /* appel API Twilio */ }
7 class AfricasTalkingProvider implements SmsProvider { /* appel API Africa's Talking */ }
8 class NovaSendSmsProvider implements SmsProvider { /* appel API NovaSend */ }
9 class SimulationSmsProvider implements SmsProvider { /* log console, aucun appel réseau */ }
10
11 @Injectable()
12 class SmsProviderFactory {
13   getProvider(): SmsProvider {
14     const selected = process.env.SMS_PROVIDER || 'simulation';
15     // ... résout primary/secondary et retourne l'implémentation (avec ou sans failover)
16   }
17 }
```

2.1 Où ce pattern est appliqué dans NovaSMS

Famille	Fichier factory	Interface Strategy
SMS	providers/sms/sms.provider.factory.ts	SmsProvider
Email	providers/email/email.provider.factory.ts	EmailProvider
WhatsApp	providers/whatsapp/whatsapp.provider.factory.ts	WhatsappProvider
Paiement	providers/payment/payment.provider.factory.ts	MobileMoneyProvider / VisaProvider
Stockage	providers/storage/storage.provider.factory.ts	StorageProvider
Push (non branché)	providers/push/push.provider.factory.ts	PushProvider

2.2 Deux façons de choisir le provider actif

- **Sélection explicite par variable** !' SMS, Email, WhatsApp, Paiement : la variable d'environnement (ex. SMS_PROVIDER=twilio) désigne directement le fournisseur voulu
- **Détection automatique par présence de configuration** !' Stockage : s'il n'y a pas de variable explicite, la factory choisit S3 automatiquement si CAMPAIGN_IMAGE_BUCKET est renseigné, sinon elle retombe sur le stockage local

3 — Canal SMS — SMS_PROVIDER

3

Variable : SMS_PROVIDER — valeur par défaut : simulation. Fichier : providers/sms/sms.provider.factory.ts.

Valeur	Fournisseur	Failover automatique
simulation	Aucun appel réseau, log console	Aucun (provider seul)
novasend	API NovaSend	Aucun (provider seul)
twilio	Twilio	Secondaire = africastalking (si configuré)
africastalking	Africa's Talking	Secondaire = twilio (si configuré)

&™ Mécanique du failover (FailoverSmsProvider)

1. Le message est envoyé au provider primaire.
2. Si le primaire répond success:false OU lève une exception, un avertissement est journalisé et le même message est immédiatement renvoyé via le provider secondaire, de façon totalement transparente pour le code appelant (campaigns, automatisations).
3. Le failover n'est activé que si le provider secondaire est réellement configuré (credentials présents en variables d'environnement) — sinon le système journalise un avertissement et fonctionne en mode "provider seul", sans jamais planter.

.env — exemples de bascule SMS

```
1 # Production Afrique de l'Ouest (recommandé)
2 SMS_PROVIDER=novasend
3 NOVASEND_SMS_API_KEY=ns_sms_XXXXXXXXXXXXXXXXXXXX
4 NOVASEND_SMS_SENDER_ID=NovaSMS
5
6 # Production avec failover Twilio -> Africa's Talking
7 SMS_PROVIDER=twilio
8 TWILIO_ACCOUNT_SID=ACXXXXXXXXXXXXXXXXXXXXXXXXXXXX
9 TWILIO_AUTH_TOKEN=XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
10 TWILIO_PHONE_NUMBER=+1XXXXXXXXXX
11 AFRICASTALKING_API_KEY=XXXXXXXXXXXXXXXXXXXXXXXXXXXX # active le fallback
12 AFRICASTALKING_USERNAME=monCompteProd
13 AFRICASTALKING_SENDER_ID=NovaSMS
```


4 — Canal Email — EMAIL_PROVIDER

Variable : EMAIL_PROVIDER — valeur par défaut : resend. Fichier : providers/email/email.provider.factory.ts.

Valeur	Fournisseur	Failover automatique
resend	Resend	Secondaire = brevo (si BREVO_API_KEY présent)
brevo	Brevo (ex-Sendinblue)	Secondaire = resend (si RESEND_API_KEY présent)
mock	Aucun appel réseau, log console	Aucun

Filet de sécurité supplémentaire : si ni Resend ni Brevo ne sont configurés (aucune clé API présente) et qu'aucun override de test n'est fourni, la factory bascule automatiquement sur MockEmailProvider plutôt que de faire planter l'envoi — utile en environnement de démonstration sans compte fournisseur actif.

5 — Canal WhatsApp — WHATSAPP_PROVIDER

Variable : WHATSAPP_PROVIDER — valeur par défaut : mock. Fichier : providers/whatsapp/whatsapp.provider.factory.ts.

- **mock** !' aucun appel réseau, utilisé par défaut et en tests
- **twilio** !' API WhatsApp Business de Twilio, activée seulement si TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN et TWILIO_WHATSAPP_NUMBER sont tous les trois présents ; sinon repli automatique sur mock avec avertissement journalisé

Particularité : contrairement aux autres canaux, WhatsApp n'a pas de mécanisme de failover entre deux fournisseurs réels — un seul fournisseur (Twilio) est implémenté à ce jour.

6 — Paiement — MOBILE_MONEY_PROVIDER & VISA_PROVIDER

Fichier : providers/payment/payment.provider.factory.ts. Deux variables indépendantes pilotent deux familles de paiement distinctes, chacune avec sa propre valeur par défaut simulation.

Variable	Valeurs possibles	Détail
MOBILE_MONEY_PROVIDER	simulation (défaut) / novasend	novasend nécessite NOVASEND_MM_API_KEY
VISA_PROVIDER	simulation (défaut) / stripe	stripe nécessite STRIPE_SECRET_KEY (préfixe sk_test_ détecté automatiquement comme sandbox)

&™ Particularité technique — singletons de simulation

Les providers de simulation (SimulationMobileMoneyProvider, SimulationVisaProvider) sont instanciés une seule fois (singleton dans la factory), afin que leur état en mémoire (transactions simulées stockées dans une Map) survive entre deux appels HTTP successifs — indispensable pour pouvoir tester un parcours complet (initiation puis confirmation) sans base de données réelle de paiement.

7 — Opérateurs Mobile Money — règles par opérateur

9

Indépendamment du choix du fournisseur technique (simulation/novasend), le module mobile-money applique des règles métier codées par opérateur local (mobile-money.service.ts, constante OPERATOR_RULES) : montant minimum/maximum en FCFA et préfixes téléphoniques valides pour reconnaître automatiquement l'opérateur à partir du numéro saisi.

Opérateur	Montant min	Montant max	Préfixes valides
WAVE	500 FCFA	500 000 FCFA	01, 05, 07, 27
ORANGE MONEY	500 FCFA	300 000 FCFA	05, 07, 25, 45, 47, 57, 65, 67, 77, 87, 97
MTN MOMO	500 FCFA	500 000 FCFA	05, 25, 45, 65
MOOV MONEY	500 FCFA	300 000 FCFA	01, 41, 61

Particularité Orange Money : un code OTP à 4 chiffres est obligatoire (vérifié par expression régulière `/^\d{4}$/`) avant toute confirmation de paiement — règle propre à cet opérateur, absente pour Wave, MoMo et Moov. Chaque opérateur a également un message de confirmation localisé affiché à l'utilisateur (ex. Wave : « Ouvrez votre application Wave pour confirmer le paiement » ; MoMo : « ... ou composez *133# »).

Q — Ajouter un cinquième opérateur Mobile Money demain (ex. Free Money) — que faut-il modifier ?

Uniquement le fichier mobile-money.service.ts : ajouter une entrée dans OPERATOR_RULES (montants + préfixes) et dans OPERATOR_MESSAGES (message de confirmation), puis étendre le type OperatorKey. Aucune modification du frontend Rechargement.tsx n'est nécessaire au-delà de l'ajout de l'icône et de l'option dans la liste déroulante — la logique de validation, elle, est entièrement pilotée par cette table centrale.

8 — Stockage fichiers — détection automatique S3 / local

10

Fichier : `providers/storage/storage.provider.factory.ts`. Contrairement aux autres familles, ce choix n'est pas piloté par une simple valeur de provider mais par la présence effective de configuration S3.

Condition	Provider retenu
<code>CAMPAIGN_IMAGE_STORAGE_PROVIDER=s3</code> (explicite)	<code>S3StorageProvider</code>
<code>CAMPAIGN_IMAGE_BUCKET</code> renseigné, pas de variable explicite	<code>S3StorageProvider</code> (détection automatique)
Aucune des deux conditions ci-dessus	<code>LocalStorageProvider</code> (fallback filesystem)

Variables S3 associées : `CAMPAIGN_IMAGE_S3_ACCESS_KEY_ID`, `CAMPAIGN_IMAGE_S3_SECRET_ACCESS_KEY`, `CAMPAIGN_IMAGE_S3_REGION` (défaut `us-east-1`), `S3_ENDPOINT` ou `CAMPAIGN_IMAGE_S3_ENDPOINT` (pour MinIO en développement), `CAMPAIGN_IMAGE_S3_FORCE_PATH_STYLE`, `CAMPAIGN_IMAGE_PUBLIC_BASE_URL`. Ce mécanisme permet d'utiliser MinIO en local (conteneur `novasms-minio` observé dans l'environnement Docker du projet) et un vrai bucket AWS S3 en production, sans changement de code.

9 — Notifications Push — PUSH_PROVIDER (non branché)

11

Variable : PUSH_PROVIDER — valeur par défaut : mock. Fichier : providers/push/push.provider.factory.ts. Fonctionnalité implémentée et testée (fcm.push.provider.spec.ts) mais — comme documenté au Doc 2, chapitre 10.4 — non référencée dans aucun module NestJS actif à ce jour. Le mode configuration existe donc déjà pour ce canal (bascule mock !" fcm), mais aucun endpoint applicatif ne l'utilise encore en production.

10 — Résilience — le Circuit Breaker maison

12

Au-dessus du choix de fournisseur, le module mobile-money protège ses appels externes avec un Circuit Breaker (common/circuit-breaker.util.ts) écrit sans dépendance externe — pattern distinct de la Factory mais complémentaire : la Factory choisit QUEL fournisseur appeler, le Circuit Breaker décide SI on doit encore l'appeler après une série d'échecs.

États du circuit

closed !' fonctionnement normal
open !' service jugé indisponible, appels bloqués
half-open !' test prudent de rétablissement

Paramètres par défaut

failureThreshold = 5 échecs
successThreshold = 2 succès
timeout = 60 secondes avant réessai

Fonctionnement : après 5 échecs consécutifs d'appel au fournisseur Mobile Money, le circuit passe en état open et toute nouvelle tentative échoue immédiatement (sans même essayer d'appeler le fournisseur) avec un message indiquant le temps d'attente restant — ce qui évite de saturer un fournisseur déjà en panne ou de faire attendre l'utilisateur final le temps d'un timeout réseau complet. Après 60 secondes, le circuit passe en half-open et laisse passer les appels suivants à titre de test ; 2 succès consécutifs referment le circuit (retour à la normale).

Ø=Üj Pourquoi ce choix plutôt qu'une librairie tierce (ex. opossum)

L'implémentation est volontairement minimale (une seule classe, ~90 lignes, zéro dépendance) car le besoin se limite à un seul point d'intégration critique (Mobile Money). Ce choix évite d'ajouter une dépendance supplémentaire au projet pour un besoin ponctuel, au prix d'une réutilisabilité un peu moindre si le même mécanisme devait être étendu demain aux autres familles de providers (SMS, Email, Visa).

11 — Procédure pratique — basculer simulation !' production

13

Checklist opérationnelle pour un déploiement réel, canal par canal, sans toucher au code applicatif.

- **1. Choisir le fournisseur SMS de production !'** renseigner SMS_PROVIDER (novasend recommandé pour l'Afrique de l'Ouest) + les identifiants associés
- **2. Choisir le fournisseur Email de production !'** renseigner EMAIL_PROVIDER=resend (ou brevo) + RESEND_API_KEY (et éventuellement BREVO_API_KEY pour activer le failover)
- **3. Activer WhatsApp si besoin !'** WHATSAPP_PROVIDER=twilio + les 3 identifiants Twilio WhatsApp
- **4. Activer le paiement réel !'** MOBILE_MONEY_PROVIDER=novasend + NOVASEND_MM_API_KEY, et VISA_PROVIDER=stripe + STRIPE_SECRET_KEY (clé sk_live_... en production, sk_test_... en recette)
- **5. Configurer le stockage !'** renseigner CAMPAIGN_IMAGE_BUCKET + les identifiants S3 pour bénéficier automatiquement du stockage S3 sans variable de choix explicite
- **6. Vérifier l'état de chaque famille de fournisseurs !'** appeler GET /campaigns/providers/health (exposé aussi dans la page Intégrations du frontend), qui renvoie pour chaque canal le fournisseur primaire actif, le fournisseur de secours et son état de configuration
- **7. Redémarrer le service backend !'** les factories lisent les variables d'environnement à l'instanciation (au démarrage du processus Node), un redémarrage est donc nécessaire après toute modification du .env

12 — Récapitulatif — toutes les variables de configuration

14

Canal	Variable de sélection	Valeurs
SMS	SMS_PROVIDER	simulation · novasend · twilio · africastalking
Email	EMAIL_PROVIDER	resend · brevo · mock
WhatsApp	WHATSAPP_PROVIDER	twilio · mock
Paieement Mobile Money	MOBILE_MONEY_PROVIDER	simulation · novasend
Paieement carte	VISA_PROVIDER	simulation · stripe
Stockage fichiers	CAMPAIGN_IMAGE_STORAGE_PROVIDER (ou auto)	s3 (auto si bucket renseigné) · local
Notifications push	PUSH_PROVIDER	mock · fcm (non branché à l'application)

Ce tableau, combiné aux endpoints de santé GET /campaigns/providers/health, constitue la référence unique pour tout futur exercice de configuration ou d'audit du mode simulation/production de la plateforme.